

课程笔记

UCB CS294/194-196 Agentic AI (Fall 2025) 第 04 讲: Post-Training Verifiable Agents

Codex

2026 年 4 月 16 日



UC Berkeley Center for Responsible,
Decentralized Intelligence

AGENTIC AI MOOC
POST-TRAINING
VERIFIABLE AGENTS
SEPTEMBER 29, 2025

Jiantao Jiao
Director of Research & Distinguished Scientist

 NVIDIA

视频作者/频道: Berkeley RDI / Agentic AI MOOC

发布日期: 2025-09-29

视频时长: 1:17:28

视频链接: <https://www.youtube.com/watch?v=3l0Zxus34es&list=PLS01nW3RtgoqGkm4UeqNeZLccW-0Gc1fJ&index=3>

目录

1 从聊天模型到可验证代理	2
1.1 讲座主线	2
1.2 环境、工具与验证器	2
1.3 本章小结	2
2 训练可验证代理需要什么	2
2.1 第一步：训练数据	2
2.2 第二步：定义 intelligence	3
2.3 第三步：把可验证数据喂进模型	3
2.4 本章小结	4
3 从训练技巧到 recipe	4
3.1 训练更久、采样更好、任务更难	4
3.2 从 benchmark 到开放问题	4
3.3 拓展阅读	4
3.4 本章小结	4
4 总结与延伸	5

1 从聊天模型到可验证代理

1.1 讲座主线

Jiantao Jiao 这讲的核心命题非常明确：早期聊天模型主要被设计成 *human aligned models*，也就是尽可能给出更符合人类偏好的回答；而 *agentic models* 则要进一步对齐 *environment feedback*，在保留人类偏好的同时，最大化可验证奖励 (*verifiable reward*)。这意味着 agent 不再只是“把话说好”，而是要在代码仓库、浏览器、数学环境、API 系统等外部环境里持续完成任务，并把任务结果交给明确的验证器。

维度	传统聊天模型	Agentic 模型
优化目标	人类偏好	人类偏好 + 可验证奖励
典型交互	用户对话	用户 + 环境 + 工具 + 验证器
典型任务	问答、闲聊、写作	编码、浏览、检索、规划、多步推理
评价方式	偏好排序、人类打分	单元测试、数学检查器、DOM 脚本、环境状态校验

表 1: 聊天模型与 agentic 模型的目标和反馈结构差异。

1.2 环境、工具与验证器

这门课把 *environment* 定义得非常具体。环境可以是代码仓库、网页浏览器、软件 API、数学题环境，甚至是多轮用户交互的模拟器。工具则是模型可调用的操作接口，它们要么扩展模型的观察能力，要么直接改变环境状态。验证器 (*verifier*) 则负责把最终输出转成奖惩信号，比如代码单元测试、数学证明检查器、网页 DOM 脚本、数据库状态检查等。

一句话理解

对 agentic 模型来说，关键不是“会不会回答”，而是“能不能在给定环境里走出一条最终可验证正确的轨迹”。这也是为什么后训练阶段要同时看环境、工具和验证器，而不能只看对话文本本身。

讲者特别强调，*conversation* 本身往往不可验证，因为很多问题没有唯一标准答案；但任务类问题常常有少数几个正确结果，错误结果却非常明显。这种差异决定了 agentic 训练不能直接沿用传统聊天模型的 reward 设计。

1.3 本章小结

这一部分建立了全课的基本坐标系：agentic models 的核心不是“更像人说话”，而是“在环境反馈里更可靠地完成任务”。后面的训练、评估和项目设计，全部围绕环境、工具和验证器展开。

2 训练可验证代理需要什么

2.1 第一步：训练数据

讲座给出的第一步非常朴素，但也最容易被低估：先收集高质量的可验证训练数据。这里的数据不是普通聊天语料，而是必须能够被验证器判定成对错的轨迹。讲者把这类数据分成三条主线：让模型理解更多样的环境，学会使用更多样的工具，并在更多样的验证器上表现良好。

训练数据的三重多样性

1. **环境多样性**: 代码仓库、网页、数学题、软件 API。
2. **工具多样性**: 浏览器、函数调用、系统命令、检索接口。
3. **验证器多样性**: 单元测试、数学检查器、DOM 校验、状态比较。

讲者还特别提醒，验证器质量本身就是训练质量的一部分。验证器不能有太多 false positives，也不能有太多 false negatives。以找零问题为例，如果答案可以写成 1/2、2/4 等多个等价形式，验证器就要奖励所有正确形式；但如果题目明确要求最简分数，验证器又必须拒绝 2/4 这种看似对、实际上不合要求的答案。

2.2 第二步：定义 intelligence

第二步并不是去“堆更多模型参数”，而是先弄清楚我们到底在测什么叫 intelligence。讲义把这件事和 benchmark 设计直接连起来：如果 benchmark 只覆盖极少数任务，就会把 intelligence 定义得太窄；如果 benchmark 太容易被 saturate，又会迅速失去测量价值。相反，一个好的 benchmark suite 应该在 hardness、diversity 和 separability 三个维度上都有足够的设计空间。

维度	关注点	设计问题
Hardness	任务难度	任务是否真的需要推理、工具或规划，而不是简单模板匹配？
Diversity	任务覆盖面	是否覆盖不同环境、工具和约束条件？
Separability	区分能力	是否能区分强模型与弱模型，而不是一上来就全都满分？

表 2: 讲座中对 benchmark quality 的三个核心维度。

2.3 第三步：把可验证数据喂进模型

在训练 recipe 上，讲者把 SFT 和 RL 放在同一条链路里理解。SFT 的作用不是“把模型训得最强”，而是让模型先学会采样出 *meaningful trajectories*，避免 RL 采样阶段充满无意义的胡乱尝试。RL 的作用则是把环境反馈和 verifier 反馈转回模型参数，让模型放大好的动作序列、压低坏的动作序列。

$$\max_{\theta} \mathbb{E}_{x \sim D, y \sim \pi_{\theta}(\cdot | x)} [r(x, y)]$$

这里的符号可以按下面理解：

- x 是输入任务或 prompt。
- y 是模型在环境中采样到的行动轨迹或输出。
- $\pi_{\theta}(\cdot | x)$ 是参数为 θ 的策略。
- $r(x, y)$ 是 verifier 给出的奖励，或者由奖励模型近似出来的分数。

讲者进一步提到一个更贴近训练经验的 proxy，可以把“高奖励轨迹”和“策略概率”之间的关系写成协方差项，直观上是鼓励模型把概率质量放到既高奖励、又足够多样的轨迹上：

$$\text{Cov}(\pi_{\theta}(y | x) A(y, x), \log \pi_{\theta}(y | x))$$

2.4 本章小结

训练可验证代理不是单点技巧，而是三件事的组合：先造出足够多样、可验证的训练数据，再把 intelligence 的定义做成有区分度的 benchmark 体系，最后用 SFT + RL recipe 把这些信号稳定地喂回模型。

3 从训练技巧到 recipe

3.1 训练更久、采样更好、任务更难

讲座后半段把很多“看起来像工程 trick 的东西”统一成了 recipe 语言。第一类是 *train longer*：让训练更稳定、更持久，核心思想不是盲目拉长训练，而是要在 reward 和 entropy 之间找到更好的平衡。第二类是 *sample better*：在 RL 采样时尽量产生更有信息量、更多样的轨迹，而不是把算力浪费在完全无意义的失败样本上。第三类是 *train harder*：把训练任务往更难、更接近真实使用场景的方向推，让模型学会处理困难 prompt，而不是只在简单样例上刷分。

三个常见 recipe

1. **Train longer**：用 entropy balance、reward/advantage shaping 让训练更稳。
2. **Sample better**：采样更有信息量的多样轨迹。
3. **Train harder**：通过更难的 prompt、反向 formulation、Pass@K to Pass@1 reduction 提升上限。

讲者反复强调，任务难度不能过高到 pass-rate 直接归零，也不能过低到没有负反馈。难度太高时，模型学不到东西；太容易时，模型只是在重复已有能力。真正有效的是让模型在“能探索出一些有用 building blocks”的区间里学习。

3.2 从 benchmark 到开放问题

最后的开放问题其实很有启发性：如果要真正把 agentic models 做强，我们需要一个更大的 collection，把 environments、evaluations、algorithms 一起开源、一起迭代。讲者提出的四个问题分别是：怎样设计这套 collection、怎样在不损害稳定性的前提下把训练重点放到某些 lessons 上、怎样平衡自主探索与教师监督、以及怎样比较不同 intelligence 定义。

3.3 拓展阅读

- SWE-bench Verified：软件修复类可验证 benchmark 的代表。
- BrowseComp：浏览型 agent 的可验证 benchmark，强调网页搜索与结果判定。

3.4 本章小结

这部分的关键信息是：可验证代理的进步不是靠单一技巧，而是靠训练时长、采样质量和任务难度的联合设计。真正难的地方不只是“做出一个能跑的 agent”，而是做出一套能持续提升 agent 的 training recipe。

4 总结与延伸

这一讲把“agentic models”从抽象概念落到了可操作的工程框架上。最重要的结论有三条：第一，agentic models 的目标函数已经从单纯的人类偏好，扩展到环境反馈和可验证奖励；第二，训练一个好 agent 不是只靠模型本身，而是要同时设计环境、工具、验证器、训练数据和 benchmark；第三，真正有效的提升方式不是刷短期分数，而是构建一个能长期演化的 benchmark 与 recipe 生态。

从教材化的角度看，这一讲也给后续课程铺了路。后面谈 evaluation 时，核心问题会变成“如何知道什么叫 works”；谈训练 agentic models 时，核心问题会变成“如何让 reward、data、exploration 和 infrastructure 协同工作”。这就是后训练代理研究的主轴。